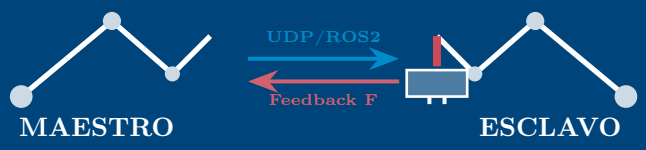


Examen Sorpresa

Peg-in-Hole Teleoperado

en Simulación por Software

TE3001B — Fundamentación de Robótica

Computational Robotics Lab
Tecnológico de Monterrey

ROBOTICS COMPUTING
Complex and Intelligent Interaction
Innovation Systems

Prof. Alberto Muñoz amunoz@tec.mx

Fecha: 23 de Febrero de 2026 **Duración:** 120 minutos
Modalidad: Equipos (dos laptops)

Inspirado en: Thompson, Reid, **Muñoz**, Murray — *IEEE Trans. Systems, Man & Cybernetics*, 2001. Oxford Robotics Group.

Contents

1 Descripción del Reto **2**
1.1 Arquitectura del sistema . . . 2
1.2 Especificaciones del robot simulado . . . 2

2 Marco Teórico **3**
2.1 Modelo dinámico del robot 3R . . . 3
2.2 Jacobiano y la dualidad velocidad–fuerza . . . 3
2.3 Computed Torque (cancelación de no-linealidades) . . . 3
2.4 Control de impedancia (esclavo en contacto) . . . 3
2.5 Mapeo maestro–esclavo . . . 4

3 Código Python — PC Maestro **4**
3.1 Simulador del robot maestro con graficación . . . 4

4 Código Python — PC Esclavo **17**
4.1 Simulador del robot esclavo con Peg-in-Hole . . . 17

5 Conexión entre dos Computadoras **30**
5.1 Pasos para conectar ambas PCs . . . 30
5.2 Comandos de ejecución . . . 30
5.3 Diagnóstico de red — herramienta de prueba . . . 30

6 Gráficas Esperadas **32**
6.1 Torques articulares durante la inserción . . . 33
6.2 Fuerzas de contacto durante la inserción . . . 33
6.3 Magnitud del error cartesiano . . . 33

7 Preguntas del Examen **34**

8 Rúbrica de Evaluación **35**

9 Instalación Rápida de Dependencias **35**

Bibliografía **36**

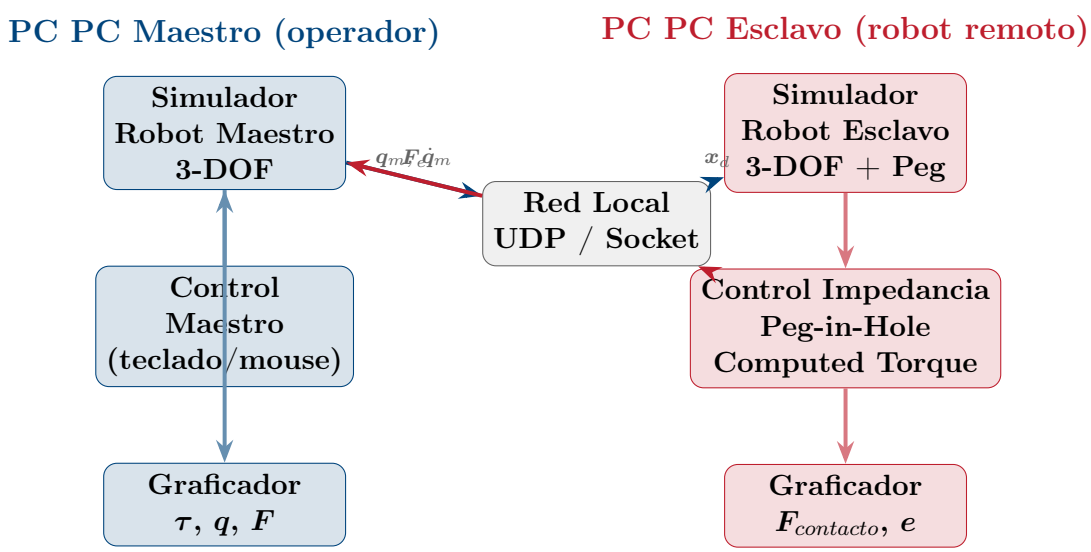
1. Descripción del Reto

(i) Contexto histórico

Hace 27 años, en el Oxford Robotics Group, el Prof. Muñoz y sus colegas construyeron un sistema de teleoperación con robots Stäubli, 3 PCs QNX a 400 MHz y un sensor de fuerza ATI de \$5,000 USD. **Hoy, con una sola laptop tienen** $\approx 10{,}000\times$ **más poder de cómputo.** El concepto es idéntico; la implementación, mucho más accesible y por ahora sin visión 3D.

En este examen implementarán un sistema completo de **teleoperación maestro–esclavo** con retroalimentación de fuerza y tarea de **Peg-in-Hole**, todo en simulación (sin hardware real). El sistema corre distribuido en **dos computadoras** conectadas por la misma red local.

1.1. Arquitectura del sistema



1.2. Especificaciones del robot simulado

Ambos robots son manipuladores planares de **3 grados de libertad** (3R) con parámetros:

Eslabón	Longitud	Masa	Rango articular
L_1	0.35 m	1.5 kg	$[-\pi/2,\ \pi/2]$
L_2	0.30 m	1.0 kg	$[-2\pi/3,\ 2\pi/3]$
L_3	0.20 m	0.5 kg	$[-\pi/2,\ \pi/2]$

El eslabón 3 termina en una **pinza** que agarra el peg (cilindro de 15 mm de diámetro). La tarea consiste en mover el peg desde su posición inicial hasta insertarlo en el soporte (agujero de 16 mm de diámetro, tolerancia de 0.5 mm).

2. Marco Teórico

2.1. Modelo dinámico del robot 3R

La ecuación de movimiento de un manipulador rígido es:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} + \boldsymbol{J}(\boldsymbol{q})^\top \boldsymbol{F}_{ext} \tag{1}$$

donde $\boldsymbol{M} \in \mathbb{R}^{3\times 3}$ es la matriz de inercia (simétrica y positiva definida), $\boldsymbol{C}$ contiene los términos de Coriolis y centrífuga, $\boldsymbol{g}$ es el par gravitacional, $\boldsymbol{\tau}$ son los torques actuados y $\boldsymbol{F}_{ext}$ es la fuerza externa en el efector final.

2.2. Jacobiano y la dualidad velocidad–fuerza

$$\dot{\boldsymbol{x}} = \boldsymbol{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}} \qquad \text{(velocidades cartesianas)} \tag{2}$$

$$\boldsymbol{\tau} = \boldsymbol{J}(\boldsymbol{q})^\top \boldsymbol{F} \qquad \text{(principio de trabajos virtuales)} \tag{3}$$

Para el robot 3R planar, el Jacobiano analítico es:

$$\boldsymbol{J}(\boldsymbol{q}) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} - L_3 s_{123} & -L_2 s_{12} - L_3 s_{123} & -L_3 s_{123} \\ L_1 c_1 + L_2 c_{12} + L_3 c_{123} & L_2 c_{12} + L_3 c_{123} & L_3 c_{123} \end{bmatrix} \tag{4}$$

con $s_{ij} = \sin(q_i + q_j)$, $c_{ij} = \cos(q_i + q_j)$.

2.3. Computed Torque (cancelación de no-linealidades)

La ley de control del Computed Torque lineariza exactamente la dinámica:

$$\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\,\boldsymbol{a}_d + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) \tag{5}$$

donde la aceleración de referencia es:

$$\boldsymbol{a}_d = \ddot{\boldsymbol{q}}_d + K_v(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) + K_p(\boldsymbol{q}_d - \boldsymbol{q}) \tag{6}$$

El sistema en lazo cerrado se reduce a un doble integrador lineal: $\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0}$.

2.4. Control de impedancia (esclavo en contacto)

Durante la inserción, el esclavo usa control de impedancia que emula un sistema masa–resorte–amortiguador:

$$M_d\ddot{\boldsymbol{e}}_x + B_d\dot{\boldsymbol{e}}_x + K_d\boldsymbol{e}_x = \boldsymbol{F}_{ext} \tag{7}$$

Los torques resultantes son:

$$\boldsymbol{\tau}_s = \boldsymbol{J}_s^\top K_d(\boldsymbol{x}_d - \boldsymbol{x}_s) + \boldsymbol{J}_s^\top B_d(\dot{\boldsymbol{x}}_d - \dot{\boldsymbol{x}}_s) \tag{8}$$

2.5. Mapeo maestro–esclavo

La referencia del esclavo sigue la posición del maestro con factor de escala α:

$$\boldsymbol{x}_d(t) = \boldsymbol{x}_{s,0} + \alpha\big(\boldsymbol{x}_m(t) - \boldsymbol{x}_{m,0}\big) \tag{9}$$

El feedback háptico al maestro es:

$$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \cdot \beta \cdot \boldsymbol{F}_e \tag{10}$$

3. Código Python — PC Maestro

3.1. Simulador del robot maestro con graficación

El siguiente código implementa la simulación completa del robot maestro: cinemática 3R, Jacobiano, Computed Torque, interfaz de control por teclado, y graficación en tiempo real de ángulos, torques y velocidad cartesiana.

```python
"""

     ROBOT MAESTRO      TE3001B  Peg-in-Hole Teleoperado
     Simulación 3-DOF + Computed Torque + Graficación
     Prof. Alberto Muñoz     Computational Robotics Lab
     Tec de Monterrey, 2026


Ejecutar en la PC MAESTRO:
    python3 master_robot.py --slave-ip <IP_DEL_ESCLAVO>

Controles:
    W/S       mover efector final en +y/-y
    A/D       mover efector final en -x/+x
    Q/E       abrir/cerrar pinza
    ESC       salir y guardar datos
"""

import numpy as np
import matplotlib
matplotlib.use('TkAgg')          # cambiar a 'Qt5Agg' si TkAgg falla
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import socket
import threading
import time
import json
import argparse

#
```

```python
# PARÁMETROS DEL ROBOT 3R
#

L1, L2, L3 = 0.35, 0.30, 0.20  # longitudes eslabones [m]
M1, M2, M3 = 1.5, 1.0, 0.5     # masas de eslabones  [kg]
G_GRAV      = 9.81              # aceleración gravitacional [m/s^2]

# Ganancias del Computed Torque
KP = np.diag([120.0, 100.0, 80.0])   # rigidez articular
KV = np.diag([25.0,  20.0,  15.0])   # amortiguamiento

# Paso de integración
DT = 0.01    # 100 Hz

#

# CINEMÁTICA DIRECTA 3R PLANAR
#

def fk_3r(q):
    """
    Cinemática directa para robot 3R planar.

    Args:
        q (np.ndarray): Vector de ángulos articulares [q1, q2, q3] en radianes.

    Returns:
        tuple: (x, y) posición del efector final en metros.
    """
    q1, q2, q3 = q
    # Posiciones de cada articulación (útil para visualización)
    x1 = L1 * np.cos(q1)
    y1 = L1 * np.sin(q1)
    x2 = x1 + L2 * np.cos(q1 + q2)
    y2 = y1 + L2 * np.sin(q1 + q2)
    x3 = x2 + L3 * np.cos(q1 + q2 + q3)
    y3 = y2 + L3 * np.sin(q1 + q2 + q3)
    return np.array([x3, y3])


def fk_3r_full(q):
    """
    Retorna las posiciones de todas las articulaciones y el EF.

    Returns:
        np.ndarray: Matriz (4x2) con posiciones [base, j1, j2, EF].
    """
```

```
    q1, q2, q3 = q
    p0 = np.array([0.0, 0.0])
    p1 = np.array([L1*np.cos(q1), L1*np.sin(q1)])
    p2 = p1 + np.array([L2*np.cos(q1+q2), L2*np.sin(q1+q2)])
    p3 = p2 + np.array([L3*np.cos(q1+q2+q3), L3*np.sin(q1+q2+q3)])
    return np.array([p0, p1, p2, p3])


#
# JACOBIANO ANALÍTICO 3R PLANAR  (J     R^{2 3 })
#
def jacobian_3r(q):
    """
    Jacobiano analítico del robot 3R planar.
    Relaciona velocidades articulares con velocidad cartesiana del EF:
            = J(q)  q

    Args:
        q (np.ndarray): Vector de ángulos articulares [q1, q2, q3].

    Returns:
        np.ndarray: Matriz Jacobiana 2 3 .
    """
    q1, q2, q3 = q
    s1   = np.sin(q1)
    s12  = np.sin(q1 + q2)
    s123 = np.sin(q1 + q2 + q3)
    c1   = np.cos(q1)
    c12  = np.cos(q1 + q2)
    c123 = np.cos(q1 + q2 + q3)

    J = np.array([
        [-L1*s1 - L2*s12 - L3*s123,
         -L2*s12 - L3*s123,
         -L3*s123],
        [ L1*c1 + L2*c12 + L3*c123,
          L2*c12 + L3*c123,
          L3*c123]
    ])
    return J


#
# MODELO DINÁMICO SIMPLIFICADO (punto de masa)
#
```

```python
def inertia_matrix(q):
    """
    Matriz de inercia M(q) para robot 3R (modelo punto de masa al extremo).
    Propiedad: simétrica y positiva definida en todo el espacio de trabajo.

    Args:
        q (np.ndarray): Ángulos articulares.

    Returns:
        np.ndarray: Matriz de inercia 3 3 .
    """
    q1, q2, q3 = q
    c2  = np.cos(q2)
    c3  = np.cos(q3)
    c23 = np.cos(q2 + q3)

    # Términos dominantes (modelo simplificado punto de masa)
    m11 = (M1*L1**2 + M2*(L1**2 + L2**2 + 2*L1*L2*c2) +
           M3*(L1**2 + L2**2 + L3**2 + 2*L1*L2*c2 +
               2*L1*L3*c23 + 2*L2*L3*c3))
    m12 = (M2*(L2**2 + L1*L2*c2) +
           M3*(L2**2 + L3**2 + L1*L2*c2 + L1*L3*c23 + 2*L2*L3*c3))
    m13 = M3*(L3**2 + L1*L3*c23 + L2*L3*c3)
    m22 = M2*L2**2 + M3*(L2**2 + L3**2 + 2*L2*L3*c3)
    m23 = M3*(L3**2 + L2*L3*c3)
    m33 = M3 * L3**2

    M = np.array([[m11, m12, m13],
                  [m12, m22, m23],
                  [m13, m23, m33]])
    return M


def coriolis_matrix(q, dq):
    """
    Matriz de Coriolis y fuerzas centrífugas C(q, q ).
    Calculada via diferenciación numérica de M(q) (método Christoffel).

    Args:
        q  (np.ndarray): Ángulos articulares.
        dq (np.ndarray): Velocidades articulares.

    Returns:
        np.ndarray: Matriz C 3 3  tal que C*q  son las fuerzas de Coriolis.
```

```python
    """
    eps = 1e-5
    n = len(q)
    M0 = inertia_matrix(q)
    C = np.zeros((n, n))

    # Símbolos de Christoffel (simplificados, solo términos dominantes)
    for k in range(n):
        qp = q.copy(); qp[k] += eps
        qm = q.copy(); qm[k] -= eps
        dM_dk = (inertia_matrix(qp) - inertia_matrix(qm)) / (2*eps)
        C += 0.5 * dM_dk * dq[k]

    return C


def gravity_vector(q):
    """
    Vector de par gravitacional g(q) para robot 3R planar.
    Asume que la gravedad actúa en dirección -y del plano.

    Args:
        q (np.ndarray): Ángulos articulares.

    Returns:
        np.ndarray: Vector de pares gravitacionales 3 1  [Nm].
    """
    q1, q2, q3 = q
    c1   = np.cos(q1)
    c12  = np.cos(q1 + q2)
    c123 = np.cos(q1 + q2 + q3)

    g1 = G_GRAV * ((M1+M2+M3)*L1*c1 + (M2+M3)*L2*c12 + M3*L3*c123)
    g2 = G_GRAV * ((M2+M3)*L2*c12 + M3*L3*c123)
    g3 = G_GRAV * M3 * L3 * c123
    return np.array([g1, g2, g3])


#
# COMPUTED TORQUE CONTROLLER
#
def computed_torque(q, dq, q_des, dq_des, ddq_des,
                    kp=KP, kv=KV, F_ext=None):
    """
    Ley de control de Computed Torque (cancelación exacta de
       no-linealidades).
```

```

    Control:      = M(q)   a    + C(q, q  )  q    + g(q)
    donde:      a    =  q      + Kv( q      -  q  ) + Kp( q    - q)

    El sistema en lazo cerrado se reduce a:    +  K v     +  K p  e  = 0

    Args:
        q        (np.ndarray): Posición articular actual [rad].
        dq       (np.ndarray): Velocidad articular actual [rad/s].
        q_des    (np.ndarray): Posición articular deseada [rad].
        dq_des   (np.ndarray): Velocidad articular deseada [rad/s].
        ddq_des  (np.ndarray): Aceleración articular deseada [rad/s  ].
        kp       (np.ndarray): Ganancia proporcional (matriz diagonal).
        kv       (np.ndarray): Ganancia derivativa (matriz diagonal).
        F_ext    (np.ndarray): Fuerza externa en EF para feedback
           háptico [N].

    Returns:
        tuple: (tau, e, de)     torques [Nm], error posición, error
           velocidad.
    """
    # Error de seguimiento
    e  = q_des  - q
    de = dq_des - dq

    # Aceleración de referencia (control PD en espacio articular)
    a_d = ddq_des + kv @ de + kp @ e

    # Matrices dinámicas
    M_mat = inertia_matrix(q)
    C_mat = coriolis_matrix(q, dq)
    g_vec = gravity_vector(q)

    # Torque de control (cancela no-linealidades)
    tau = M_mat @ a_d + C_mat @ dq + g_vec

    # Agregar feedback háptico si hay fuerza externa medida
    if F_ext is not None and np.linalg.norm(F_ext) > 0.01:
        J = jacobian_3r(q)
        tau += J.T @ F_ext    #  _fb  =  J             F

    # Saturar torques para seguridad
    tau = np.clip(tau, -20.0, 20.0)
    return tau, e, de


#
```

```python
# INTEGRADOR EULER (dinámica del robot)
#

def integrate_dynamics(q, dq, tau, dt=DT):
    """
    Integra la dinámica del robot usando Euler explícito.
    Ecuación: M(q)  q   =    - C(q, q  )  q   - g(q)

    Args:
        q   (np.ndarray): Posición articular actual.
        dq  (np.ndarray): Velocidad articular actual.
        tau (np.ndarray): Vector de torques aplicados.
        dt  (float):      Paso de integración [s].

    Returns:
        tuple: (q_new, dq_new)     nueva posición y velocidad articular.
    """
    M_mat = inertia_matrix(q)
    C_mat = coriolis_matrix(q, dq)
    g_vec = gravity_vector(q)

    # Aceleración articular resultante
    ddq = np.linalg.solve(M_mat, tau - C_mat @ dq - g_vec)

    # Integración Euler
    dq_new = dq + ddq * dt
    q_new  = q  + dq_new * dt

    # Aplicar límites articulares
    q_limits = np.array([np.pi/2, 2*np.pi/3, np.pi/2])
    q_new  = np.clip(q_new,  -q_limits, q_limits)
    dq_new = np.clip(dq_new, -3.0, 3.0)   # límite de velocidad [rad/s]
    return q_new, dq_new


#

# COMUNICACIÓN DE RED     MAESTRO (CLIENTE)
#

class MasterNetClient:
    """
    Cliente UDP del maestro. Envía posición cartesiana al esclavo
    y recibe fuerzas de contacto para feedback háptico.

    Protocolo JSON: {"xd": [x, y], "gripper": bool}
    Respuesta:      {"Fe": [Fx, Fy], "contact": bool}
    """
```

```python
    def __init__(self, slave_ip, port_tx=9001, port_rx=9002):
        self.slave_ip = slave_ip
        self.port_tx  = port_tx
        self.port_rx  = port_rx
        self.sock_tx  = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock_rx  = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock_rx.bind(('', self.port_rx))
        self.sock_rx.settimeout(0.005)   # no bloquear el loop de control
        self.Fe       = np.zeros(2)      # fuerza recibida del esclavo [N]
        self.contact  = False
        self._thread  = threading.Thread(target=self._recv_loop, daemon=True)
        self._thread.start()

    def send_command(self, xd, gripper=True):
        """Envía posición deseada al esclavo."""
        msg = json.dumps({"xd": xd.tolist(), "gripper": int(gripper)})
        self.sock_tx.sendto(msg.encode(), (self.slave_ip, self.port_tx))

    def _recv_loop(self):
        """Hilo receptor     actualiza Fe y estado de contacto."""
        while True:
            try:
                data, _ = self.sock_rx.recvfrom(256)
                parsed  = json.loads(data.decode())
                self.Fe      = np.array(parsed["Fe"])
                self.contact = bool(parsed["contact"])
            except (socket.timeout, json.JSONDecodeError):
                pass


#
# CLASE PRINCIPAL     ROBOT MAESTRO
#
class MasterRobot:
    """
    Simulador completo del robot maestro con:
    - Dinámica 3R con Computed Torque
    - Control cartesiano por teclado
    - Feedback háptico desde el esclavo
    - Graficación en tiempo real (4 subplots)
    - Registro de datos para análisis
    """
    def __init__(self, slave_ip="127.0.0.1"):
```

```
        # Estado inicial del robot
        self.q  = np.array([0.4, -0.3, 0.2])     # posición articular [rad]
        self.dq = np.zeros(3)                     # velocidad articular
        self.q_des  = self.q.copy()
        self.dq_des = np.zeros(3)
        self.ddq_des = np.zeros(3)

        # Velocidad de movimiento cartesiano (controlado por teclado)
        self.v_cart = np.zeros(2)
        self.v_step = 0.05  # [m/s] por pulsación de tecla

        # Red
        self.net = MasterNetClient(slave_ip)

        # Histórico para graficación (buffer circular de 500 muestras)
        N = 500
        self.hist_t    = np.zeros(N)
        self.hist_q    = np.zeros((N, 3))
        self.hist_tau  = np.zeros((N, 3))
        self.hist_Fe   = np.zeros((N, 2))
        self.hist_x    = np.zeros((N, 2))
        self.idx = 0
        self.t   = 0.0

        # Estado de la pinza
        self.gripper_open = False

        # Referencia cinemática inversa (DLS)
        self.q_des = self.q.copy()

    def ik_dls(self, x_des, damp=0.01):
        """
        Cinemática inversa numérica por Damped Least Squares (DLS).
        Actualiza q_des para seguir la posición cartesiana deseada.

        Args:
            x_des (np.ndarray): Posición cartesiana deseada [m].
            damp  (float):      Factor de amortiguamiento (evita singularidades).
        """
        for _ in range(5):   # iteraciones internas
            x_cur = fk_3r(self.q_des)
            e_x   = x_des - x_cur
            if np.linalg.norm(e_x) < 1e-4:
                break
            J     = jacobian_3r(self.q_des)
            # Pseudoinversa DLS: J^+ =  J  ( J J   +   I  )^{-1}
```

```
            JJT  = J @ J.T
            Jp   = J.T @ np.linalg.inv(JJT + damp**2 * np.eye(2))
            self.q_des = self.q_des + Jp @ e_x

    def step(self):
        """
        Ejecuta un paso de simulación (dt = DT):
        1. Actualizar referencia cartesiana
        2. IK para obtener q_des
        3. Computed Torque
        4. Integrar dinámica     q, q
        5. Enviar comando al esclavo
        6. Registrar datos
        """
        # 1. Referencia cartesiana = posición actual + v_cart*dt
        x_cur = fk_3r(self.q)
        x_des = x_cur + self.v_cart * DT

        # 2. Cinemática inversa
        self.ik_dls(x_des)

        # 3. Computed Torque con feedback háptico
        tau, e, de = computed_torque(
            self.q, self.dq, self.q_des, self.dq_des, self.ddq_des,
            F_ext = 0.3 * self.net.Fe  # factor    de feedback háptico
        )

        # 4. Integrar dinámica
        self.q, self.dq = integrate_dynamics(self.q, self.dq, tau)

        # 5. Enviar posición cartesiana al esclavo
        x_ef = fk_3r(self.q)
        self.net.send_command(x_ef, gripper=not self.gripper_open)

        # 6. Registrar datos para gráficas
        i = self.idx % 500
        self.hist_t[i]   = self.t
        self.hist_q[i]   = self.q
        self.hist_tau[i] = tau
        self.hist_Fe[i]  = self.net.Fe
        self.hist_x[i]   = x_ef
        self.idx += 1
        self.t   += DT


#

# GRAFICACIÓN EN TIEMPO REAL
```

```python
#

def setup_plots(robot):
    """
    Configura la figura de graficación con 4 paneles:
    1. Vista 2D del robot (cinemática)
    2. Torques articulares      ,      ,       vs tiempo
    3. Fuerzas de contacto Fx, Fy vs tiempo
    4. Ángulos articulares  q  ,  q  ,  q   vs tiempo
    """
    fig = plt.figure(figsize=(14, 10),
                     facecolor='#0a0a1a')
    fig.suptitle('TE3001B     Robot Maestro 3R | Peg-in-Hole Teleoperado',
                 color='white', fontsize=14, fontweight='bold', y=0.98)

    # Paleta de colores
    C = ['#00BFFF', '#FF6B6B', '#69FF47', '#FFD700', '#FF69B4', '#00FFD0']
    bg = '#0d1117'
    grid_c = '#1e2530'

    ax_robot = fig.add_subplot(2, 2, 1, facecolor=bg)
    ax_tau   = fig.add_subplot(2, 2, 2, facecolor=bg)
    ax_force = fig.add_subplot(2, 2, 3, facecolor=bg)
    ax_q     = fig.add_subplot(2, 2, 4, facecolor=bg)

    for ax in [ax_robot, ax_tau, ax_force, ax_q]:
        ax.tick_params(colors='#aaa')
        ax.xaxis.label.set_color('#aaa')
        ax.yaxis.label.set_color('#aaa')
        ax.title.set_color('white')
        for spine in ax.spines.values():
            spine.set_edgecolor('#333')
        ax.grid(True, color=grid_c, linestyle='--', alpha=0.5)

    # Panel 1: Robot 2D
    ax_robot.set_xlim(-0.9, 0.9)
    ax_robot.set_ylim(-0.9, 0.9)
    ax_robot.set_aspect('equal')
    ax_robot.set_title('Vista Cinemática 3R', fontsize=11)
    ax_robot.set_xlabel('x [m]'); ax_robot.set_ylabel('y [m]')
    link_line, = ax_robot.plot([], [], 'o-', color=C[0],
                               linewidth=3, markersize=8,
                                  markerfacecolor=C[1])
    ef_dot,    = ax_robot.plot([], [], 's', color=C[2], markersize=12,
                               markerfacecolor=C[3], zorder=5)
    # Dibujar agujero destino (HOLE)
```

```
    hole_x, hole_y = 0.55, 0.10
    ax_robot.add_patch(plt.Circle((hole_x, hole_y), 0.025,
                                  color='#FFD700', alpha=0.6))
    ax_robot.plot(hole_x, hole_y, 'x', color='white', markersize=8,
       markeredgewidth=2)
    ax_robot.text(hole_x+0.03, hole_y+0.03, 'HOLE', color='#FFD700',
       fontsize=8)

    # Panel 2: Torques articulares
    ax_tau.set_title('Torques Articulares    [Nm]', fontsize=11)
    ax_tau.set_xlabel('Tiempo [s]'); ax_tau.set_ylabel('   [Nm]')
    lines_tau = [ax_tau.plot([], [], color=C[i], linewidth=1.5,
                             label=f'  {i+1}')[0] for i in range(3)]
    ax_tau.legend(loc='upper right', fontsize=9,
                  facecolor='#1a1a2e', labelcolor='white')
    ax_tau.axhline(y=0, color='#444', linewidth=0.8)

    # Panel 3: Fuerzas de contacto
    ax_force.set_title('Fuerzas de Contacto Reflejadas [N]',
       fontsize=11)
    ax_force.set_xlabel('Tiempo [s]'); ax_force.set_ylabel('F [N]')
    line_Fx, = ax_force.plot([], [], color=C[4], linewidth=1.8,
       label='Fx')
    line_Fy, = ax_force.plot([], [], color=C[5], linewidth=1.8,
       label='Fy')
    ax_force.legend(loc='upper right', fontsize=9,
                    facecolor='#1a1a2e', labelcolor='white')
    ax_force.axhline(y=0, color='#444', linewidth=0.8)

    # Panel 4: Ángulos articulares
    ax_q.set_title('Ángulos Articulares q [rad]', fontsize=11)
    ax_q.set_xlabel('Tiempo [s]'); ax_q.set_ylabel('q [rad]')
    lines_q = [ax_q.plot([], [], color=C[i], linewidth=1.5,
                         label=f'q{i+1}',
                            linestyle=['solid','dashed','dotted'][i])[0]
               for i in range(3)]
    ax_q.legend(loc='upper right', fontsize=9,
                facecolor='#1a1a2e', labelcolor='white')

    plt.tight_layout(rect=[0, 0.02, 1, 0.96])

    return fig, (ax_robot, ax_tau, ax_force, ax_q), \
           (link_line, ef_dot), lines_tau, (line_Fx, line_Fy), lines_q


def main(slave_ip):
    robot = MasterRobot(slave_ip)
    fig, axes, arm_lines, lines_tau, force_lines, lines_q =
```

```
        setup_plots(robot)
    ax_robot, ax_tau, ax_force, ax_q = axes
    link_line, ef_dot = arm_lines
    line_Fx, line_Fy  = force_lines

    # Hilo de simulación en background
    running = [True]
    def sim_loop():
        while running[0]:
            robot.step()
            time.sleep(DT)
    sim_thread = threading.Thread(target=sim_loop, daemon=True)
    sim_thread.start()

    def animate(frame):
        n  = min(robot.idx, 500)
        i0 = robot.idx % 500
        # Reordenar buffer circular
        idx_range = np.arange(i0, i0 + n) % 500
        t   = robot.hist_t[idx_range]
        tau = robot.hist_tau[idx_range]
        Fe  = robot.hist_Fe[idx_range]
        q_h = robot.hist_q[idx_range]

        # Cinemática actual
        pts = fk_3r_full(robot.q)
        link_line.set_data(pts[:,0], pts[:,1])
        ef_dot.set_data([pts[-1,0]], [pts[-1,1]])

        # Ventana de tiempo visible (últimos 5 s)
        t_win = 5.0
        mask  = (t > robot.t - t_win) if robot.t > t_win else
           np.ones(n, bool)

        for i, ln in enumerate(lines_tau):
            ln.set_data(t[mask], tau[mask, i])
        ax_tau.set_xlim(max(0, robot.t - t_win), max(t_win, robot.t))
        ax_tau.relim(); ax_tau.autoscale_view(scalex=False)

        line_Fx.set_data(t[mask], Fe[mask, 0])
        line_Fy.set_data(t[mask], Fe[mask, 1])
        ax_force.set_xlim(max(0, robot.t - t_win), max(t_win, robot.t))
        ax_force.relim(); ax_force.autoscale_view(scalex=False)

        for i, ln in enumerate(lines_q):
            ln.set_data(t[mask], q_h[mask, i])
        ax_q.set_xlim(max(0, robot.t - t_win), max(t_win, robot.t))
        ax_q.relim(); ax_q.autoscale_view(scalex=False)
```

```python

        return [link_line, ef_dot] + lines_tau + [line_Fx, line_Fy] +
            lines_q

    # Captura de teclas (movimiento cartesiano)
    def on_key_press(event):
        v = robot.v_step
        if   event.key == 'w': robot.v_cart = np.array([0, v])
        elif event.key == 's': robot.v_cart = np.array([0, -v])
        elif event.key == 'd': robot.v_cart = np.array([v, 0])
        elif event.key == 'a': robot.v_cart = np.array([-v, 0])
        elif event.key == 'q': robot.gripper_open = True
        elif event.key == 'e': robot.gripper_open = False
        elif event.key == 'escape':
            running[0] = False
            plt.close()

    def on_key_release(event):
        robot.v_cart = np.zeros(2)

    fig.canvas.mpl_connect('key_press_event', on_key_press)
    fig.canvas.mpl_connect('key_release_event', on_key_release)

    ani = animation.FuncAnimation(fig, animate, interval=50,   # 20 fps
                                  blit=False, cache_frame_data=False)
    plt.show()


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="TE3001B     Robot Maestro Peg-in-Hole")
    parser.add_argument("--slave-ip", default="127.0.0.1",
                        help="IP del PC esclavo (default: loopback)")
    args = parser.parse_args()
    main(args.slave_ip)
```

Listing 1: Robot maestro 3R — simulación y control

4. Código Python — PC Esclavo

4.1. Simulador del robot esclavo con Peg-in-Hole

```python
"""

     ROBOT ESCLAVO     TE3001B  Peg-in-Hole Teleoperado
     Control de Impedancia + Detección de Contacto
```

```python
     Graficación de fuerzas de contacto y torques


Ejecutar en la PC ESCLAVO:
    python3 slave_robot.py

Nodos de red:
    Escucha en UDP:9001 (recibe xd del maestro)
    Envía  en UDP:9002 al maestro (Fe, estado)
"""

import numpy as np
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import socket, threading, time, json, argparse

#                                   parámetros del robot (idénticos al maestro)

L1, L2, L3 = 0.35, 0.30, 0.20
M1, M2, M3 = 1.5, 1.0, 0.5
G_GRAV      = 9.81
DT          = 0.01

#                                   ganancias de impedancia

KD_IMP = 400.0   # rigidez virtual [N/m]        Kd de la impedancia
BD_IMP = 40.0    # amortiguamiento virtual [N s/m]
KP_ART = np.diag([100.0, 80.0, 60.0])   # ganancias Computed Torque
KV_ART = np.diag([20.0,  16.0, 12.0])

#                                   geometría del Peg-in-Hole

PEG_LENGTH  = 0.08   # longitud del peg [m]
PEG_RADIUS  = 0.008  # radio del peg  [m]
HOLE_CENTER = np.array([0.55, 0.10])   # posición del agujero en espacio global
HOLE_RADIUS = 0.009  # radio del agujero (tolerancia 1mm)
F_CONTACT_K = 2000.0  # rigidez de contacto pared del agujero [N/m]
F_THRESHOLD = 2.0     # umbral de detección de contacto [N]


#

# FUNCIONES CINEMÁTICAS (reutilizadas del maestro)
```

```python
#

def fk_3r(q):
    q1, q2, q3 = q
    x1 = L1*np.cos(q1); y1 = L1*np.sin(q1)
    x2 = x1 + L2*np.cos(q1+q2); y2 = y1 + L2*np.sin(q1+q2)
    x3 = x2 + L3*np.cos(q1+q2+q3); y3 = y2 + L3*np.sin(q1+q2+q3)
    return np.array([x3, y3])

def fk_3r_full(q):
    q1, q2, q3 = q
    p0 = np.array([0.0, 0.0])
    p1 = np.array([L1*np.cos(q1), L1*np.sin(q1)])
    p2 = p1 + np.array([L2*np.cos(q1+q2), L2*np.sin(q1+q2)])
    p3 = p2 + np.array([L3*np.cos(q1+q2+q3), L3*np.sin(q1+q2+q3)])
    return np.array([p0, p1, p2, p3])

def jacobian_3r(q):
    q1, q2, q3 = q
    s1   = np.sin(q1); s12  = np.sin(q1+q2); s123 = np.sin(q1+q2+q3)
    c1   = np.cos(q1); c12  = np.cos(q1+q2); c123 = np.cos(q1+q2+q3)
    return np.array([
        [-L1*s1-L2*s12-L3*s123, -L2*s12-L3*s123, -L3*s123],
        [ L1*c1+L2*c12+L3*c123,  L2*c12+L3*c123,  L3*c123]
    ])

def inertia_matrix(q):
    q1, q2, q3 = q
    c2 = np.cos(q2); c3 = np.cos(q3); c23 = np.cos(q2+q3)
    m11 = (M1*L1**2 + M2*(L1**2+L2**2+2*L1*L2*c2) +
           M3*(L1**2+L2**2+L3**2+2*L1*L2*c2+2*L1*L3*c23+2*L2*L3*c3))
    m12 = M2*(L2**2+L1*L2*c2) +
       M3*(L2**2+L3**2+L1*L2*c2+L1*L3*c23+2*L2*L3*c3)
    m13 = M3*(L3**2+L1*L3*c23+L2*L3*c3)
    m22 = M2*L2**2 + M3*(L2**2+L3**2+2*L2*L3*c3)
    m23 = M3*(L3**2+L2*L3*c3)
    m33 = M3*L3**2
    return np.array([[m11,m12,m13],[m12,m22,m23],[m13,m23,m33]])

def coriolis_matrix(q, dq):
    eps = 1e-5
    C = np.zeros((3,3))
    for k in range(3):
        qp = q.copy(); qp[k] += eps
        qm = q.copy(); qm[k] -= eps
        dM = (inertia_matrix(qp) - inertia_matrix(qm)) / (2*eps)
        C += 0.5 * dM * dq[k]
    return C
```

```python

def gravity_vector(q):
    q1, q2, q3 = q
    c1 = np.cos(q1); c12 = np.cos(q1+q2); c123 = np.cos(q1+q2+q3)
    g1 = G_GRAV*((M1+M2+M3)*L1*c1 + (M2+M3)*L2*c12 + M3*L3*c123)
    g2 = G_GRAV*((M2+M3)*L2*c12 + M3*L3*c123)
    g3 = G_GRAV*M3*L3*c123
    return np.array([g1, g2, g3])

def integrate_dynamics(q, dq, tau, dt=DT):
    M_mat = inertia_matrix(q)
    C_mat = coriolis_matrix(q, dq)
    g_vec = gravity_vector(q)
    ddq = np.linalg.solve(M_mat, tau - C_mat@dq - g_vec)
    dq_new = np.clip(dq + ddq*dt, -3.0, 3.0)
    q_lim  = np.array([np.pi/2, 2*np.pi/3, np.pi/2])
    q_new  = np.clip(q + dq_new*dt, -q_lim, q_lim)
    return q_new, dq_new


#
# MODELO DE CONTACTO PEG-IN-HOLE
#
class PegHoleContact:
    """
    Modelo de contacto elástico para la inserción Peg-in-Hole.

    El peg está adherido al efector final. Se detecta contacto cuando
    el peg intenta entrar al agujero con desalineación mayor al umbral.
    La fuerza de contacto es proporcional a la penetración en la pared.

    Fases de la tarea:
        APPROACH      el peg se aproxima al agujero desde arriba
        CONTACT       el peg toca el borde del agujero (Fz > umbral)
        INSERTION     control mixto: Fx, F y 0  (centrar) +  z (insertar)
        COMPLETE      inserción exitosa
    """

    APPROACH, CONTACT, INSERTION, COMPLETE = 0, 1, 2, 3
    STATE_NAMES = {0: "APROXIMACIÓN", 1: "CONTACTO",
                   2: "INSERCIÓN", 3: "COMPLETADO    "}

    def __init__(self):
        self.phase    = self.APPROACH
        self.depth    = 0.0        # profundidad de inserción [m]
```

```python

    def compute_contact_force(self, x_ef):
        """
        Calcula la fuerza de contacto ejercida sobre el peg.

        Modelo: resorte lineal con la pared del agujero.
        Si el peg está dentro del radio del agujero     sin contacto lateral.
        Si está fuera     fuerza proporcional a la penetración.

        Args:
            x_ef (np.ndarray): Posición del efector final [x, y] en metros.

        Returns:
            tuple: (F_contact [N], state_str, in_contact bool)
        """
        delta = x_ef - HOLE_CENTER          # vector EF     centro agujero
        dist  = np.linalg.norm(delta)       # distancia al centro

        F_contact = np.zeros(2)
        in_contact = False

        # Proximidad vertical: el peg está cerca del agujero
        if abs(x_ef[1] - HOLE_CENTER[1]) < PEG_LENGTH * 1.5:

            if dist < HOLE_RADIUS:
                # EF dentro del agujero     sin fuerza lateral, solo inserción
                self.phase    = max(self.phase, self.INSERTION)
                self.depth    = HOLE_CENTER[1] - x_ef[1]   # profundidad
                if self.depth > PEG_LENGTH * 0.85:
                    self.phase = self.COMPLETE
            elif dist < HOLE_RADIUS + 0.02:
                # Contacto con la pared del agujero     fuerza de reacción
                penetration   = dist - HOLE_RADIUS
                F_mag         = F_CONTACT_K * penetration
                F_contact     = -F_mag * (delta / dist)  # hacia el centro
                in_contact    = True
                if self.phase < self.CONTACT:
                    self.phase = self.CONTACT

        return F_contact, self.STATE_NAMES[self.phase], in_contact


```

```python
#

# CONTROL DE IMPEDANCIA EN ESPACIO DE TAREA
#

def impedance_control(q, dq, x_des, dx_des, q_des_prev,
                      F_contact=None, kd=KD_IMP, bd=BD_IMP):
    """
    Control de impedancia para el esclavo en contacto.

    Emula el comportamiento masa-resorte-amortiguador:
        Md   _x + Bd   _x + Kd e_x = F_ext

    Los torques se calculan vía Jacobiano transpuesto:
         _imp   =  J  (Kd e_x + Bd   _x)

    Sobre estos torques se aplica compensación de gravedad
    y Coriolis para cancelar no-linealidades.

    Args:
        q, dq        : Estado articular actual.
        x_des        : Posición cartesiana deseada (del maestro).
        dx_des       : Velocidad cartesiana deseada.
        q_des_prev   : q_des del paso anterior (para derivada numérica).
        F_contact    : Fuerza de contacto medida [N].
        kd, bd       : Rigidez y amortiguamiento virtuales.

    Returns:
        tuple: (tau, F_total, e_x)     torques, fuerza resultante,
           error cartesiano.
    """
    x_cur  = fk_3r(q)
    J      = jacobian_3r(q)

    # Velocidad cartesiana actual
    dx_cur = J @ dq

    # Error cartesiano
    e_x  = x_des - x_cur
    de_x = dx_des - dx_cur

    # Fuerza de impedancia (ley de resorte-amortiguador)
    F_imp = kd * e_x + bd * de_x

    # Incluir fuerza de contacto si la hay
    F_total = F_imp.copy()
    if F_contact is not None:
        F_total += F_contact
```

```python

    # Torques de impedancia:    =  J      F_total
    tau_imp = J.T @ F_total

    # Compensación de gravedad (siempre necesaria)
    g_vec = gravity_vector(q)
    C_mat = coriolis_matrix(q, dq)
    tau   = tau_imp + g_vec + C_mat @ dq

    # Saturar torques
    tau = np.clip(tau, -20.0, 20.0)
    return tau, F_total, e_x


#
# SERVIDOR DE RED     ESCLAVO
#
class SlaveNetServer:
    """
    Servidor UDP del esclavo.
    Recibe comandos xd del maestro y envía fuerzas de contacto de vuelta.
    """
    def __init__(self, master_ip="127.0.0.1", port_rx=9001, port_tx=9002):
        self.master_ip = master_ip
        self.port_rx   = port_rx
        self.port_tx   = port_tx
        self.sock      = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(('', self.port_rx))
        self.sock.settimeout(0.005)
        self.x_des    = np.array([0.55, 0.40])  # posición inicial deseada
        self.gripper  = True
        self._thread  = threading.Thread(target=self._recv_loop, daemon=True)
        self._thread.start()

    def _recv_loop(self):
        """Hilo receptor     actualiza xd continuamente."""
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
                parsed = json.loads(data.decode())
                self.x_des   = np.array(parsed["xd"])
```

```python
                self.gripper = bool(parsed["gripper"])
                self.master_addr = addr   # guardar para responder
            except (socket.timeout, json.JSONDecodeError, AttributeError):
                pass

    def send_force(self, Fe, contact, master_port=9002):
        """Envía fuerza de contacto al maestro."""
        msg = json.dumps({"Fe": Fe.tolist(), "contact": int(contact)})
        try:
            self.sock.sendto(msg.encode(), (self.master_ip, master_port))
        except Exception:
            pass


#
# CLASE PRINCIPAL     ROBOT ESCLAVO
#
class SlaveRobot:
    """
    Simulador completo del robot esclavo:
    - Dinámica 3R con control de impedancia
    - Modelo de contacto Peg-in-Hole
    - Graficación: fuerzas de contacto, torques, trayectoria EF
    - Comunicación bidireccional con el maestro
    """
    def __init__(self, master_ip="127.0.0.1"):
        # Estado inicial: robot extendido hacia el agujero
        self.q  = np.array([0.6, -0.5, 0.1])
        self.dq = np.zeros(3)
        self.q_des_prev = self.q.copy()

        # Modelos
        self.contact_model = PegHoleContact()
        self.net = SlaveNetServer(master_ip)

        # Histórico (buffer circular)
        N = 500
        self.hist_t      = np.zeros(N)
        self.hist_tau    = np.zeros((N, 3))
        self.hist_Fc     = np.zeros((N, 2))
        self.hist_x      = np.zeros((N, 2))
        self.hist_ex     = np.zeros((N, 2))
        self.idx = 0
        self.t   = 0.0
```

```
        self.contact_state = "APROXIMACIÓN"

    def ik_dls(self, x_des, damp=0.01):
        """Cinemática inversa por DLS     igual que en el maestro."""
        q = self.q.copy()
        for _ in range(8):
            e = x_des - fk_3r(q)
            if np.linalg.norm(e) < 1e-4:
                break
            J  = jacobian_3r(q)
            Jp = J.T @ np.linalg.inv(J @ J.T + damp**2 * np.eye(2))
            q  = q + Jp @ e
        return q

    def step(self):
        """Un paso de simulación del esclavo."""
        x_des = self.net.x_des.copy()

        # Calcular fuerza de contacto según modelo Peg-in-Hole
        x_ef = fk_3r(self.q)
        F_contact, state_str, in_contact = \
            self.contact_model.compute_contact_force(x_ef)

        self.contact_state = state_str

        # Velocidad cartesiana deseada (diferenciación numérica)
        dx_des = (x_des - fk_3r(self.ik_dls(x_des))) / DT * 0.0  # se
            usa 0

        # Control de impedancia con contacto
        tau, F_total, e_x = impedance_control(
            self.q, self.dq, x_des, dx_des,
            self.q_des_prev, F_contact
        )

        # Integrar dinámica
        self.q, self.dq = integrate_dynamics(self.q, self.dq, tau)

        # Enviar fuerza de contacto al maestro (para feedback háptico)
        self.net.send_force(F_contact, in_contact)

        # Registrar datos
        i = self.idx % 500
        self.hist_t[i]   = self.t
        self.hist_tau[i] = tau
        self.hist_Fc[i]  = F_contact
        self.hist_x[i]   = x_ef
        self.hist_ex[i]  = e_x
```

```python
        self.idx += 1
        self.t   += DT


#

# GRAFICACIÓN ESCLAVO
#

def setup_slave_plots(robot):
    """
    Configura 4 paneles para el esclavo:
    1. Vista 2D del robot + peg + agujero
    2. Fuerzas de contacto Fx, Fy vs tiempo
    3. Torques articulares
    4. Error cartesiano |e_x|, |e_y|
    """
    fig = plt.figure(figsize=(14, 10), facecolor='#0a0a0f')
    fig.suptitle('TE3001B     Robot Esclavo 3R | Control de Impedancia + Peg-in-Hole',
                 color='white', fontsize=14, fontweight='bold', y=0.98)
    bg = '#0d1117'
    C  = ['#FF6B6B', '#69FF47', '#00BFFF', '#FFD700', '#FF69B4', '#00FFD0']

    ax_robot = fig.add_subplot(2, 2, 1, facecolor=bg)
    ax_force = fig.add_subplot(2, 2, 2, facecolor=bg)
    ax_tau   = fig.add_subplot(2, 2, 3, facecolor=bg)
    ax_err   = fig.add_subplot(2, 2, 4, facecolor=bg)

    for ax in [ax_robot, ax_force, ax_tau, ax_err]:
        ax.tick_params(colors='#aaa')
        for spine in ax.spines.values(): spine.set_edgecolor('#333')
        ax.grid(True, color='#1e2530', linestyle='--', alpha=0.5)
        ax.title.set_color('white')
        ax.xaxis.label.set_color('#aaa'); ax.yaxis.label.set_color('#aaa')

    # Panel 1: Robot + escenario
    ax_robot.set_xlim(-0.9, 0.9); ax_robot.set_ylim(-0.5, 1.0)
    ax_robot.set_aspect('equal')
    ax_robot.set_title('Esclavo     Peg-in-Hole', fontsize=11)
    ax_robot.set_xlabel('x [m]'); ax_robot.set_ylabel('y [m]')
    # Dibujar agujero
    hole_patch = plt.Rectangle(
        (HOLE_CENTER[0]-0.04, HOLE_CENTER[1]-0.005), 0.08, 0.04,
        color='#2a2a4a', zorder=1)
    ax_robot.add_patch(hole_patch)
```

```
    ax_robot.plot(*HOLE_CENTER, 'x', color='#FFD700', markersize=10,
                  markeredgewidth=2, zorder=3)
    ax_robot.text(HOLE_CENTER[0]+0.02, HOLE_CENTER[1]+0.015,
                  'HOLE', color='#FFD700', fontsize=8)

    link_line, = ax_robot.plot([], [], 'o-', color=C[2], linewidth=3,
                               markersize=8, markerfacecolor=C[0])
    peg_line,  = ax_robot.plot([], [], '-', color=C[3], linewidth=5, zorder=4)
    state_text  = ax_robot.text(0.02, 0.96, '', transform=ax_robot.transAxes,
                               color='#FFD700', fontsize=10, fontweight='bold',
                               verticalalignment='top')
    force_arrow = ax_robot.annotate('', xy=(0, 0), xytext=(0, 0),
                                   arrowprops=dict(arrowstyle='->', color='red',
                                                   lw=2.5), zorder=5)

    # Panel 2: Fuerzas de contacto
    ax_force.set_title('Fuerzas de Contacto [N]', fontsize=11)
    ax_force.set_xlabel('Tiempo [s]'); ax_force.set_ylabel('F [N]')
    line_Fx, = ax_force.plot([], [], color=C[4], linewidth=2.0, label='Fx contacto')
    line_Fy, = ax_force.plot([], [], color=C[5], linewidth=2.0, label='Fy contacto')
    ax_force.axhline(y=0, color='#444', linewidth=0.8)
    ax_force.axhline(y=F_THRESHOLD, color='#FF4444', linewidth=1.2,
                     linestyle='--', label=f'Umbral {F_THRESHOLD} N')
    ax_force.legend(loc='upper right', fontsize=8,
                    facecolor='#1a1a2e', labelcolor='white')

    # Panel 3: Torques
    ax_tau.set_title('Torques Articulares    [Nm]', fontsize=11)
    ax_tau.set_xlabel('Tiempo [s]'); ax_tau.set_ylabel('   [Nm]')
    lines_tau = [ax_tau.plot([], [], color=C[i], linewidth=1.5,
                             label=f'  {i+1}')[0] for i in range(3)]
    ax_tau.axhline(y=0, color='#444', linewidth=0.8)
    ax_tau.legend(loc='upper right', fontsize=8,
                  facecolor='#1a1a2e', labelcolor='white')

    # Panel 4: Error cartesiano
    ax_err.set_title('Error Cartesiano |e| [mm]', fontsize=11)
    ax_err.set_xlabel('Tiempo [s]'); ax_err.set_ylabel('Error [mm]')
    line_ex, = ax_err.plot([], [], color=C[0], linewidth=1.8, label='| e  |')
    line_ey, = ax_err.plot([], [], color=C[1], linewidth=1.8, label='|e_y|')
```

```
    ax_err.axhline(y=1.0, color='#888', linewidth=1.0,
                   linestyle=':', label='1 mm (meta)')
    ax_err.legend(loc='upper right', fontsize=8,
                  facecolor='#1a1a2e', labelcolor='white')

    plt.tight_layout(rect=[0, 0.02, 1, 0.96])
    return (fig,
            (ax_robot, ax_force, ax_tau, ax_err),
            (link_line, peg_line, state_text, force_arrow),
            (line_Fx, line_Fy),
            lines_tau,
            (line_ex, line_ey))


def main(master_ip):
    robot = SlaveRobot(master_ip)
    (fig, axes, robot_artists, force_lines, lines_tau, err_lines) = \
        setup_slave_plots(robot)
    ax_robot, ax_force, ax_tau, ax_err = axes
    link_line, peg_line, state_text, force_arrow = robot_artists
    line_Fx, line_Fy = force_lines
    line_ex, line_ey = err_lines

    running = [True]
    def sim_loop():
        while running[0]:
            robot.step()
            time.sleep(DT)
    threading.Thread(target=sim_loop, daemon=True).start()

    def animate(frame):
        n   = min(robot.idx, 500)
        i0  = robot.idx % 500
        idx = np.arange(i0, i0+n) % 500
        t   = robot.hist_t[idx]
        tau = robot.hist_tau[idx]
        Fc  = robot.hist_Fc[idx]
        ex  = robot.hist_ex[idx]
        t_win = 5.0
        mask  = (t > robot.t - t_win)

        # Robot
        pts = fk_3r_full(robot.q)
        link_line.set_data(pts[:,0], pts[:,1])
        # Peg visual
        ef = pts[-1]
        peg_dir = pts[-1] - pts[-2]
        if np.linalg.norm(peg_dir) > 0:
```

```
            peg_dir = peg_dir / np.linalg.norm(peg_dir)
        peg_start = ef
        peg_end   = ef + peg_dir * PEG_LENGTH
        peg_line.set_data([peg_start[0], peg_end[0]],
                          [peg_start[1], peg_end[1]])
        state_text.set_text(f"Estado: {robot.contact_state}")

        # Fuerzas
        ax_force.set_xlim(max(0, robot.t-t_win), max(t_win, robot.t))
        line_Fx.set_data(t[mask], Fc[mask, 0])
        line_Fy.set_data(t[mask], Fc[mask, 1])
        ax_force.relim(); ax_force.autoscale_view(scalex=False)

        # Torques
        ax_tau.set_xlim(max(0, robot.t-t_win), max(t_win, robot.t))
        for i, ln in enumerate(lines_tau):
            ln.set_data(t[mask], tau[mask, i])
        ax_tau.relim(); ax_tau.autoscale_view(scalex=False)

        # Error (en mm)
        ax_err.set_xlim(max(0, robot.t-t_win), max(t_win, robot.t))
        line_ex.set_data(t[mask], np.abs(ex[mask, 0])*1000)
        line_ey.set_data(t[mask], np.abs(ex[mask, 1])*1000)
        ax_err.relim(); ax_err.autoscale_view(scalex=False)

        return ([link_line, peg_line] + lines_tau + [line_Fx, line_Fy,
                                                     line_ex, line_ey])

    ani = animation.FuncAnimation(fig, animate, interval=50,
                                  blit=False, cache_frame_data=False)
    plt.show()


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="TE3001B     Robot
       Esclavo")
    parser.add_argument("--master-ip", default="127.0.0.1",
                        help="IP del PC maestro (default: loopback)")
    args = parser.parse_args()
    main(args.master_ip)
```

Listing 2: Robot esclavo 3R — control de impedancia + Peg-in-Hole

5. Conexión entre dos Computadoras

Ambas laptops deben estar conectadas a la **misma red WiFi** (o por cable al mismo switch). **No usar hotspot del celular**: la mayoría usa NAT y bloquea UDP entre dispositivos.

5.1. Pasos para conectar ambas PCs

1. Conectar ambas laptops a la **misma red WiFi**
2. En PC Esclavo: `ip addr` (Linux) o `ipconfig` (Windows)
   Anotar su IP: p.ej. `192.168.1.105`
3. Verificar conectividad: `ping 192.168.1.105`
   desde la PC Maestro (debe responder)
4. Abrir puertos en firewall:
   `sudo ufw allow 9001/udp && sudo ufw allow 9002/udp`
5. Ejecutar primero el **esclavo**, luego el **maestro**

5.2. Comandos de ejecución

PC Esclavo (ejecutar primero):

```
python3 slave_robot.py --master-ip 192.168.1.100
```

PC Maestro (ejecutar después):

```
python3 master_robot.py --slave-ip 192.168.1.105
```

Prueba local (una sola PC):

```
python3 slave_robot.py &
python3 master_robot.py
```

5.3. Diagnóstico de red — herramienta de prueba

```
"""
net_test.py     Prueba de conectividad UDP entre maestro y esclavo.
Ejecutar en AMBAS computadoras para verificar la red antes del examen.
```

```python

En PC-A (receptor):   python3 net_test.py --mode server
En PC-B (emisor):     python3 net_test.py --mode client --ip <IP-PC-A>
"""

import socket, time, argparse, statistics

PORT = 9999
N_PACKETS = 100   # número de paquetes de prueba


def run_server():
    """
    Servidor UDP: recibe paquetes y mide latencia de ida y vuelta (RTT).
    Responde inmediatamente con un eco para que el cliente mida RTT.
    """
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('', PORT))
    print(f"[SERVIDOR] Esperando en puerto {PORT}...")
    count = 0
    while count < N_PACKETS:
        data, addr = sock.recvfrom(128)
        # Eco inmediato     el cliente mide el RTT
        sock.sendto(data, addr)
        count += 1
    print(f"[SERVIDOR] {count} paquetes procesados. ¡Conectividad OK!")
    sock.close()


def run_client(server_ip):
    """
    Cliente UDP: envía paquetes de prueba y mide RTT.
    Calcula estadísticas: mínimo, máximo, promedio, desviación estándar.
    """
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(1.0)   # timeout de 1 segundo por paquete
    rtts = []
    lost = 0

    print(f"[CLIENTE] Enviando {N_PACKETS} paquetes a
       {server_ip}:{PORT}...")
    for i in range(N_PACKETS):
        payload = f"PING_{i:04d}_{time.time():.6f}".encode()
        t0 = time.perf_counter()
        sock.sendto(payload, (server_ip, PORT))
        try:
            resp, _ = sock.recvfrom(128)
            rtt_ms = (time.perf_counter() - t0) * 1000.0
```

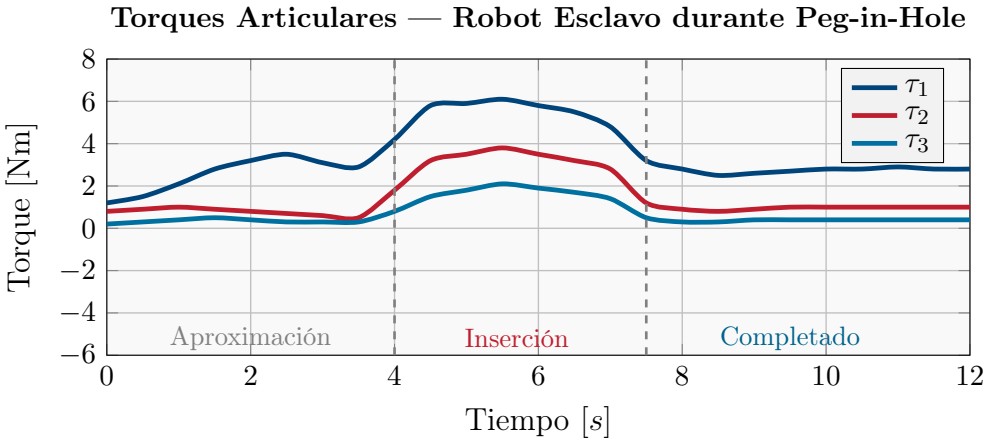
```
            rtts.append(rtt_ms)
        except socket.timeout:
            lost += 1
        time.sleep(0.01)    # 100 Hz

    sock.close()
    if rtts:
        print(f"\n{'='*50}")
        print(f"  Paquetes enviados  : {N_PACKETS}")
        print(f"  Paquetes perdidos  : {lost} ({100*lost/N_PACKETS:.1f}%)")
        print(f"  RTT mínimo         : {min(rtts):.2f} ms")
        print(f"  RTT máximo         : {max(rtts):.2f} ms")
        print(f"  RTT promedio       : {statistics.mean(rtts):.2f} ms")
        print(f"  Desv. estándar RTT : {statistics.stdev(rtts):.2f} ms")
        print(f"{'='*50}")
        # Evaluación para control de tiempo real
        if statistics.mean(rtts) < 10.0 and lost/N_PACKETS < 0.01:
            print("      RED APTA para control de impedancia (< 10 ms, < 1% pérdida)")
        else:
            print("      RED INADECUADA     revisar conexión WiFi o usar cable")
    else:
        print("ERROR: Sin respuesta del servidor. Verificar IP y firewall.")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Prueba de red UDP")
    parser.add_argument("--mode", choices=["server", "client"],
                        required=True, help="Modo de ejecución")
    parser.add_argument("--ip", default="127.0.0.1",
                        help="IP del servidor (solo en modo cliente)")
    args = parser.parse_args()
    if args.mode == "server":
        run_server()
    else:
        run_client(args.ip)
```

Listing 3: Herramienta de diagnóstico de red (net_test.py)

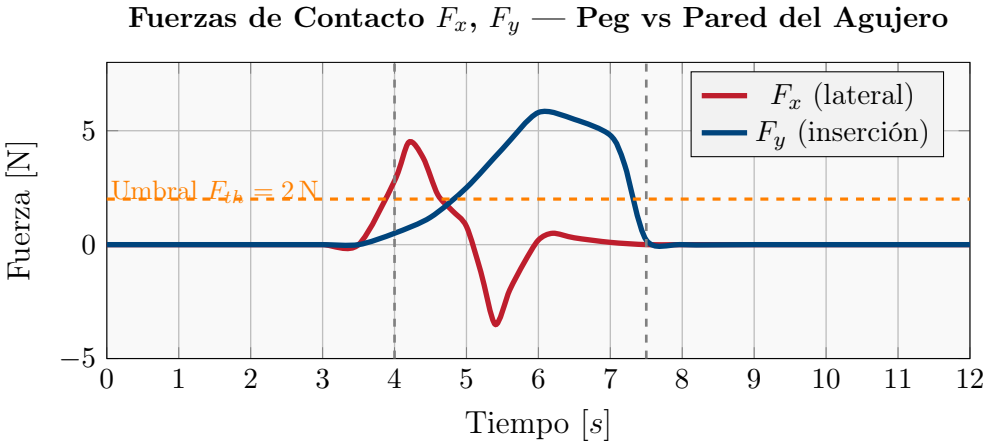
6. Gráficas Esperadas

A continuación se muestran las gráficas que el sistema debe producir durante una ejecución exitosa del reto.

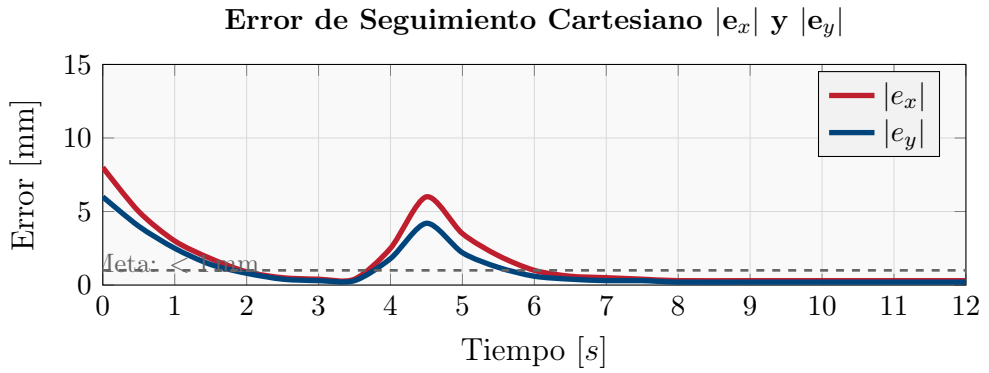
6.1. Torques articulares durante la inserción



6.2. Fuerzas de contacto durante la inserción



6.3. Magnitud del error cartesiano



7. Preguntas del Examen

(✓) Instrucciones

Respondan las siguientes preguntas **en su reporte escrito**. Incluyan capturas de pantalla de sus gráficas reales (no las del ejemplo). Tiempo total: **120 minutos**.

Parte A — Preguntas Teóricas (40%)

A1. [8 pts] Deriven analíticamente el Jacobiano $\boldsymbol{J}(\boldsymbol{q})$ del robot 3R planar implementado. ¿En qué configuraciones es singular $\boldsymbol{J}$? Expliquen qué ocurre con la estimación de fuerzas $\hat{\boldsymbol{F}} = (\boldsymbol{J}^\top)^+\boldsymbol{\tau}$ cerca de esas singularidades.

A2. [8 pts] Demuestren que el sistema bajo Computed Torque resulta en la dinámica de error lineal:

$$\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0}$$

¿Qué condiciones sobre K_p y K_v garantizan la estabilidad asintótica? Expresen la respuesta en términos de la frecuencia natural ω_n y el factor de amortiguamiento ζ.

A3. [8 pts] En el control de impedancia, K_d es la *rigidez virtual*. ¿Qué efecto tiene aumentar K_d en: (a) la sensación háptica del operador, (b) la estabilidad del sistema maestro–esclavo con retardo T_d, y (c) la velocidad de inserción del peg?

A4. [8 pts] Expliquen la ecuación de feedback háptico:

$$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \cdot \beta \cdot \boldsymbol{F}_e$$

¿Qué representa el factor β? ¿Cuál es el valor máximo de β antes de que el sistema maestro se vuelva inestable? Justifiquen con la condición de pasividad.

A5. [8 pts] Un paquete UDP se pierde el 5% del tiempo con una latencia promedio de 15 ms. Usen el criterio de Nyquist y el margen de fase para argumentar si esta red es apta para controlar con $K_d = 500$ N/m. ¿Qué modificación harían al protocolo para hacerlo más robusto?

Parte B — Análisis de Resultados (35%)

B1. [10 pts] Muestren la captura de pantalla de las **4 gráficas** de su ejecución real (maestro y esclavo). Identifiquen en las curvas de torque los instantes de: aproximación libre, primer contacto, inserción activa y completado.

B2. [10 pts] Del registro de $\boldsymbol{F}_{contacto}(t)$:

- ¿Cuál fue la fuerza lateral máxima observada?
- ¿En qué momento se detectó el contacto? ($F > F_{th}$)
- ¿Cuánto tiempo tardó la fase de inserción?

- ¿El error cartesiano final fue < 1 mm?

B3. [15 pts] Calculen el **vector Jacobiano transpuesto** $\boldsymbol{J}^\top$ para la configuración articular de su robot en el instante de primer contacto. Luego verifiquen que:

$$\boldsymbol{\tau}_{esperado} = \boldsymbol{J}^\top \boldsymbol{F}_{contacto}$$

coincide (en orden de magnitud) con lo observado en su gráfica de torques.

Parte C — Implementación y Demo (25%)

C1. [10 pts] Conecten las dos laptops y demuestren el sistema funcionando en red real (no loopback). Muestren la salida de `net_test.py` y el RTT medido.

C2. [15 pts] Demo en vivo: El operador (maestro) debe guiar el peg a la posición del agujero y completar la inserción en menos de **60 segundos**, *sin ver la pantalla del esclavo*, guiado únicamente por las gráficas de fuerza del maestro. Graben en video el momento en que el estado pasa a `"COMPLETADO "`.

8. Rúbrica de Evaluación

Criterio	Pts.	Mín. aceptable	Evidencia
Teoría (A1–A5)	40	24	Reporte escrito
Análisis de gráficas (B1–B3)	35	20	Capturas reales
Demo en red real (C1)	10	6	Salida net_test
Demo Peg-in-Hole (C2)	15	9	Video ≤ 60 s
Total	**100**	**60**	

Score ≥ 60 pts + Demo funcional en red real.
El Jacobiano transpuesto debe verse en sus torques. Si los torques no cambian durante el contacto, algo está mal en la implementación.

9. Instalación Rápida de Dependencias

```
# Dependencias Python (instalar en AMBAS computadoras)
pip install numpy matplotlib

# Verificar versiones recomendadas
python3 -c "import numpy, matplotlib; \
            print(numpy.__version__, matplotlib.__version__)"
# Esperado: numpy >= 1.24, matplotlib >= 3.7
```

```
# Abrir puertos de red (Linux/Ubuntu)
sudo ufw allow 9001/udp
sudo ufw allow 9002/udp
sudo ufw allow 9999/udp   # para net_test.py
```

(i) Tips de depuración

- Si la ventana de matplotlib no aparece: cambiar `matplotlib.use('TkAgg')` por `'Qt5Agg'` o `'Agg'` (sin ventana interactiva).
- Si hay error de `socket.timeout`: verificar que el esclavo esté corriendo *antes* que el maestro.
- Si los torques explotan (> 20 Nm saturados): reducir K_p a la mitad.
- Si la IK diverge: reducir `v_step` a 0.02 m/s.
- Para modo loopback (una PC): usar `-slave-ip 127.0.0.1` y `-master-ip 127.0.0.1`.

Bibliografía

[1] Thompson, S., Reid, I., **Muñoz, A.**, Murray, D.W. "Using a stationary camera to improve the performance of a teleoperated robot." *IEEE Trans. Systems, Man & Cybernetics-A*, Vol. 31(1), pp. 74–82, 2001.

[2] Siciliano, B., Sciavicco, L., Villani, L., Oriolo, G. *Robotics: Modelling, Planning and Control*. Springer, 2009. (Caps. 7–9: Dinámica y control de fuerza.)

[3] Spong, M.W., Hutchinson, S., Vidyasagar, M. *Robot Modeling and Control*. Wiley, 2006. (Cap. 8: Computed Torque.)

[4] Hogan, N. "Impedance Control: An Approach to Manipulation." *ASME J. Dyn. Sys., Meas., Control*, Vol. 107, 1985.

[5] Murray, R.M., Li, Z., Sastry, S.S. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994. Disponible en: http://www.cds.caltech.edu/~murray/mlswiki

[6] Niemeyer, G., Preusche, C., Hirzinger, G. "Telerobotics." En: *Springer Handbook of Robotics*, Siciliano & Khatib (eds.), Springer, 2008.

"Lo que Oxford tardó años — ustedes pueden hacerlo en clases."

Prof. Alberto Muñoz, Computational Robotics Lab, Tecnológico de Monterrey, 2026.